Université Abdelmalek Essaadi
Faculté des Sciences et Techniques de Tanger
Département Génie Informatique
LICENCE | Développement web avancé
Back end (Python)| 2025/2026

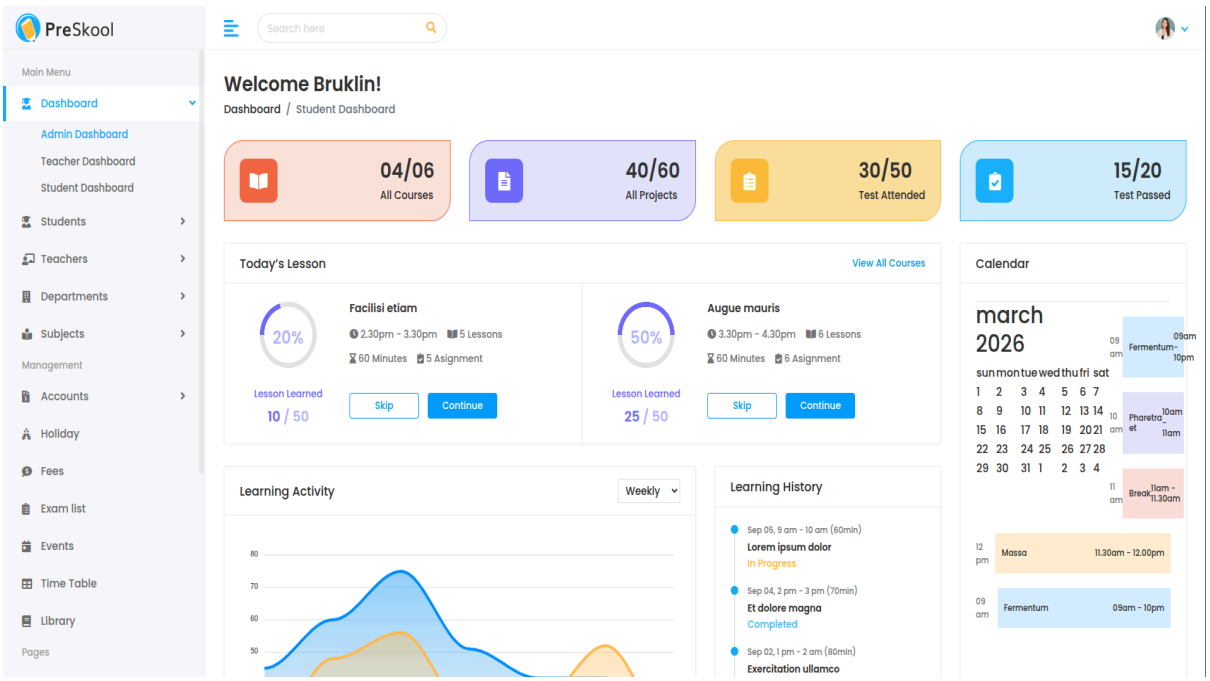
RAPORT PFM-PYTHON

REALISER PAR:

- **BENGOUROU HOUSSAM**
- **CHENAYTI YASSINE**

ENCADREE PAR:

- **Prof. Sara AHSAIN**



1.INTRODUCTION

Contexte du projet

Ce projet s'inscrit dans le cadre de la conception et du développement d'une application web nommée **« PreSkool »**, destinée à la gestion globale des opérations d'un établissement scolaire ou universitaire. Le but de ce travail était de partir d'une configuration Django de base pour évoluer vers une plateforme web complète, robuste et modulaire. Cette transition a nécessité la structuration du projet en plusieurs applications indépendantes (telles que la gestion des facultés, des étudiants et de l'authentification), l'intégration de templates **frontend** (Bootstrap), ainsi que la conception d'une architecture de base de données relationnelle.

Objectifs principaux

Pour répondre aux besoins de gestion de l'établissement, le développement de l'application s'est articulé autour des objectifs fondamentaux suivants :

- **Gestion des étudiants et de la scolarité :** Permettre l'enregistrement, la modification et le suivi des dossiers des étudiants, tout en établissant une liaison directe avec les profils de leurs parents.

- **Administration du corps professoral :** Gérer de manière centralisée les profils des enseignants et leurs informations professionnelles.

- **Organisation académique :** Structurer l'établissement à travers la création et la gestion des départements et des matières enseignées.

- **Sécurité et contrôle d'accès :** Mettre en œuvre un système d'authentification robuste et personnalisé, basé sur des rôles stricts (Administrateur, Enseignant, Étudiant). Ce système garantit que chaque utilisateur n'accède qu'aux tableaux de bord et aux fonctionnalités qui correspondent à ses privilèges.

2. Architecture et Conception

2.1 Architecture MVT (Model-View-Template)

L'application repose sur le framework Django et respecte le design pattern **MVT**, ce qui permet une séparation nette des responsabilités :

- **Model (Modèle) :** Gère la structure des données et la logique métier. Chaque application (étudiant, enseignant, etc.) possède ses propres modèles qui interagissent avec la base de données via l'ORM de Django.

- **View (Vue) :** Contient la logique de contrôle. Elle reçoit les requêtes HTTP, traite les données via les modèles et sélectionne le template approprié pour la réponse. Nous avons utilisé des vues basées sur les fonctions et les classes pour une flexibilité maximale.

- **Template (Gabarit) :** S'occupe de l'interface utilisateur (UI). En utilisant le moteur de rendu de Django, nous avons intégré des fichiers HTML dynamiques stylisés avec Bootstrap, permettant un affichage responsive et une séparation claire entre le backend et le frontend.

2.2 Diagramme de Classes (UML / Schéma de base de données)

La conception de la base de données est le cœur du projet. Voici les relations clés implémentées dans le code :

- **`CustomUser` :** Étend la classe **`AbstractUser`** de Django pour inclure des attributs de rôle spécifiques : **`is_admin`**, **`is_teacher`**, et **`is_student`**.

- **`Teacher` (Enseignant) :** Lié au **`CustomUser`** via une relation **`OneToOneField`**. Il est également associé à un **`Department`**.

- **`Student` (Étudiant) :** Possède une relation **`OneToOneField`** avec le modèle **`Parent`**. Il est inscrit dans un **`Department`**.

- **`Subject` (Matière) :** Lié à un **`Department`** et assigné à un **`Teacher`**.

VOICI LES DIAGRAMMES DE CLASSES :
Diagramme représentons le model **student :**

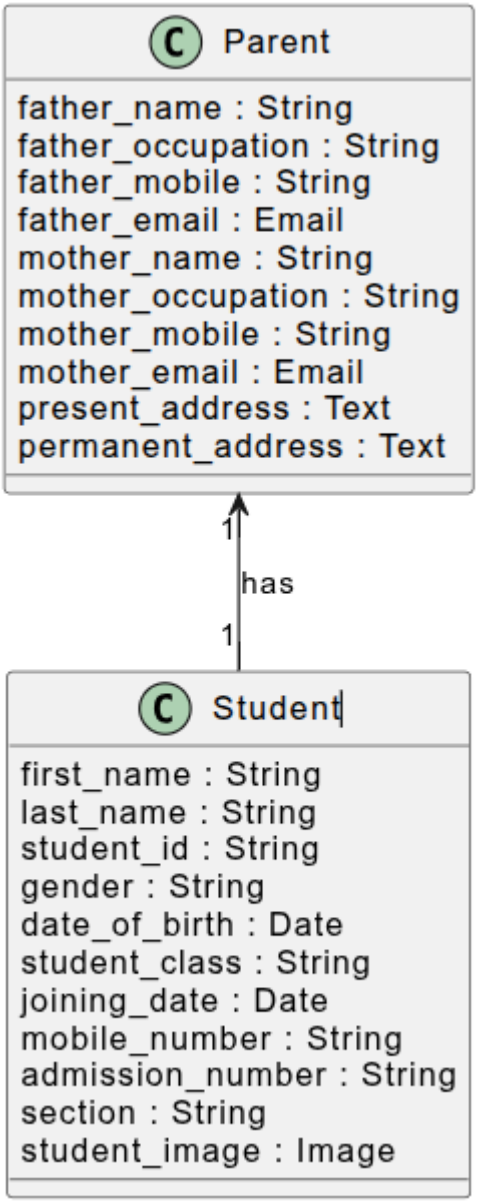
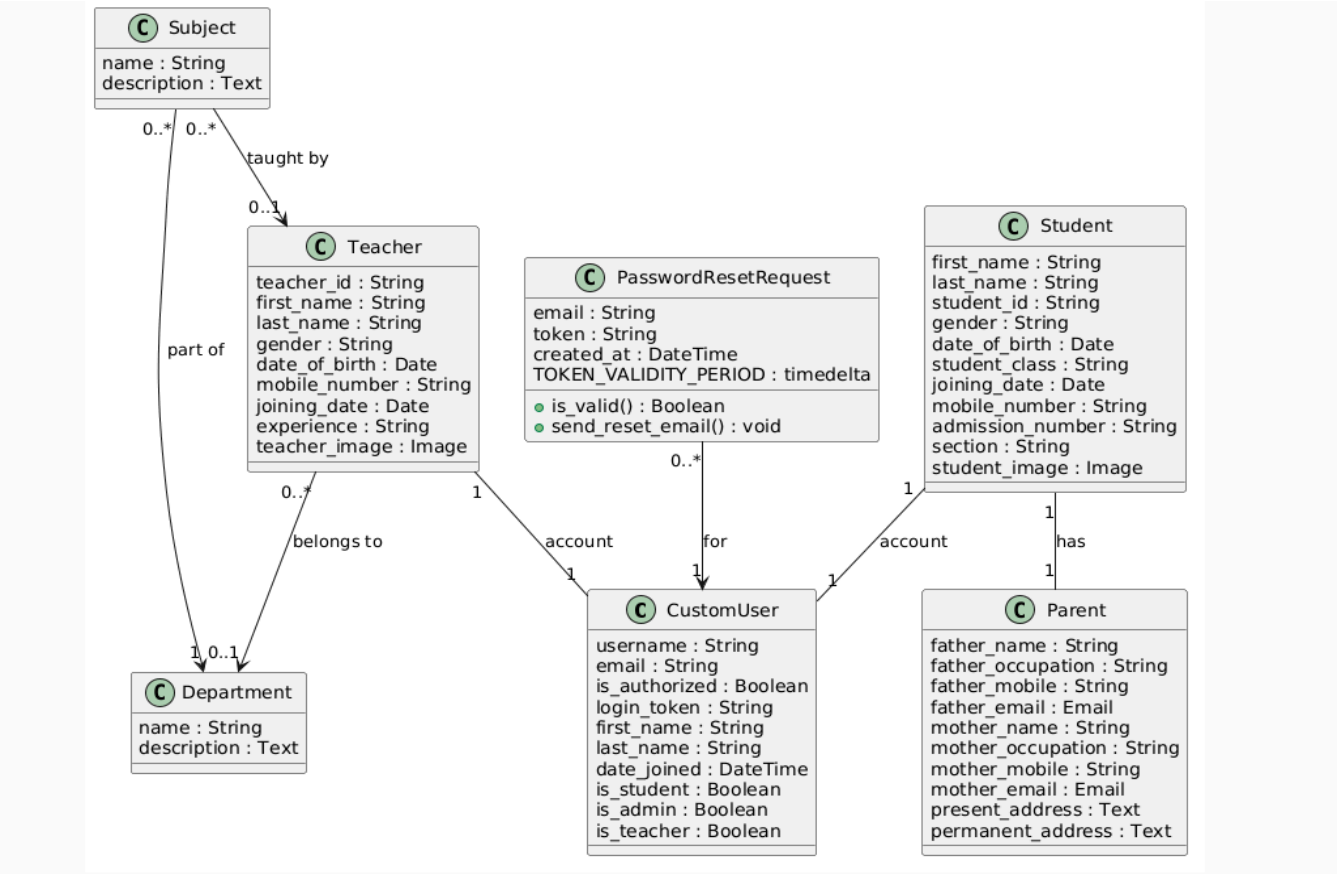


Diagramme représentons la liaisons entre les différents classes(models)
Teacher , department , subject et home_auth :



2.3 Diagramme de Cas d'Utilisation (Use Case)

Le système distingue trois acteurs principaux avec des privilèges hiérarchisés :

1.Administrateur:

- Gestion complète des utilisateurs (Ajouter/Modifier/Supprimer Enseignants et Étudiants).
- Configuration de la structure académique (Facultés, Départements, Matières).
- Gestion du calendrier (Emploi du temps, Jours fériés).

2.Enseigniant:

- Consultation de son propre emploi du temps.
- Gestion des examens et saisie des notes pour ses matières.
- Accès à la liste des étudiants de son département.

2.Etudiant:

- Consultation de son tableau de bord personnel.
- Visualisation de son emploi du temps et des dates d'examens.
- Consultation de ses résultats académiques.

3.Structure du Projet et Implémentation

Pour garantir un code propre, maintenable et évolutif (scalable), le projet **PreSkool** a été architecturé selon le principe de « séparation des responsabilités » (Separation of Concerns). Le système global a été divisé en plusieurs applications Django indépendantes et modulaires, chacune gérant une logique métier spécifique.

3.1 Découpage Modulaire

Le projet est composé des applications principales suivantes :

- **L'application `home_auth` (Authentification et Identité) :** C'est le noyau de la sécurité du système. Cette application est exclusivement responsable de la gestion du modèle utilisateur personnalisé (`CustomUser`). Elle prend en charge l'intégralité du cycle de vie des sessions utilisateurs : la connexion (login), la déconnexion (logout), ainsi que les processus sécurisés de réinitialisation de mot de passe.

- **L'application `student` (Gestion des Étudiants) :** Cette application encapsule toute la logique liée à la scolarité. Elle implémente les opérations CRUD (Création, Lecture, Mise à jour, Suppression) permettant de gérer les dossiers des étudiants et de les lier de manière cohérente aux profils de leurs parents.

- **Les applications `teacher`, `department` et `faculty` (Organisation Académique) :** Ces modules travaillent de concert pour modéliser la structure de l'établissement. Ils gèrent le personnel académique (enseignants), la création et la gestion des départements, ainsi que l'infrastructure globale (facultés). Cette séparation permet, par exemple, de modifier la logique de gestion des départements sans impacter le fonctionnement des profils enseignants.

3.2 Sécurité et Gestion des Rôles (Contrôle d'Accès)

L'une des exigences majeures du projet était de sécuriser la plateforme en fonction des profils des utilisateurs. L'approche choisie repose sur un contrôle d'accès basé sur les rôles (Role-Based Access Control - RBAC).

Au lieu d'utiliser le modèle utilisateur standard de Django de manière générique, nous avons défini des drapeaux (flags) booléens directement dans le modèle `CustomUser` : `is_admin`, `is_teacher` et `is_student`.

La restriction d'accès est implémentée au niveau des vues (Views) :

- **Vérification de l'authentification :** L'accès à la majorité de la plateforme nécessite d'être connecté (utilisation du décorateur `@login_required`).

- **Vérification des privilèges :** Avant de permettre à un utilisateur de visualiser un tableau de bord spécifique ou d'exécuter une action (comme ajouter une matière ou saisir des notes), le système évalue son rôle. Par exemple, à l'aide de décorateurs personnalisés ou de vérifications internes dans les vues, le système s'assure que `request.user.is_admin == True` pour accéder au panneau d'administration.

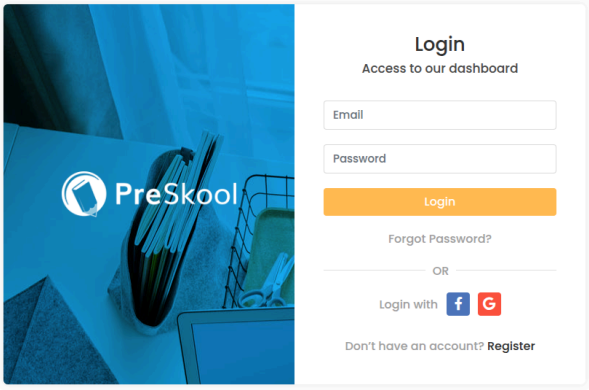
- **Étanchéité des profils :** Si un étudiant tente d'accéder à l'URL du tableau de bord d'un enseignant ou de l'administrateur, l'accès lui est refusé (redirection ou erreur 403 Forbidden), garantissant ainsi la confidentialité et l'intégrité des données de l'établissement.

4.Captures d'Écran et Fonctionnalités

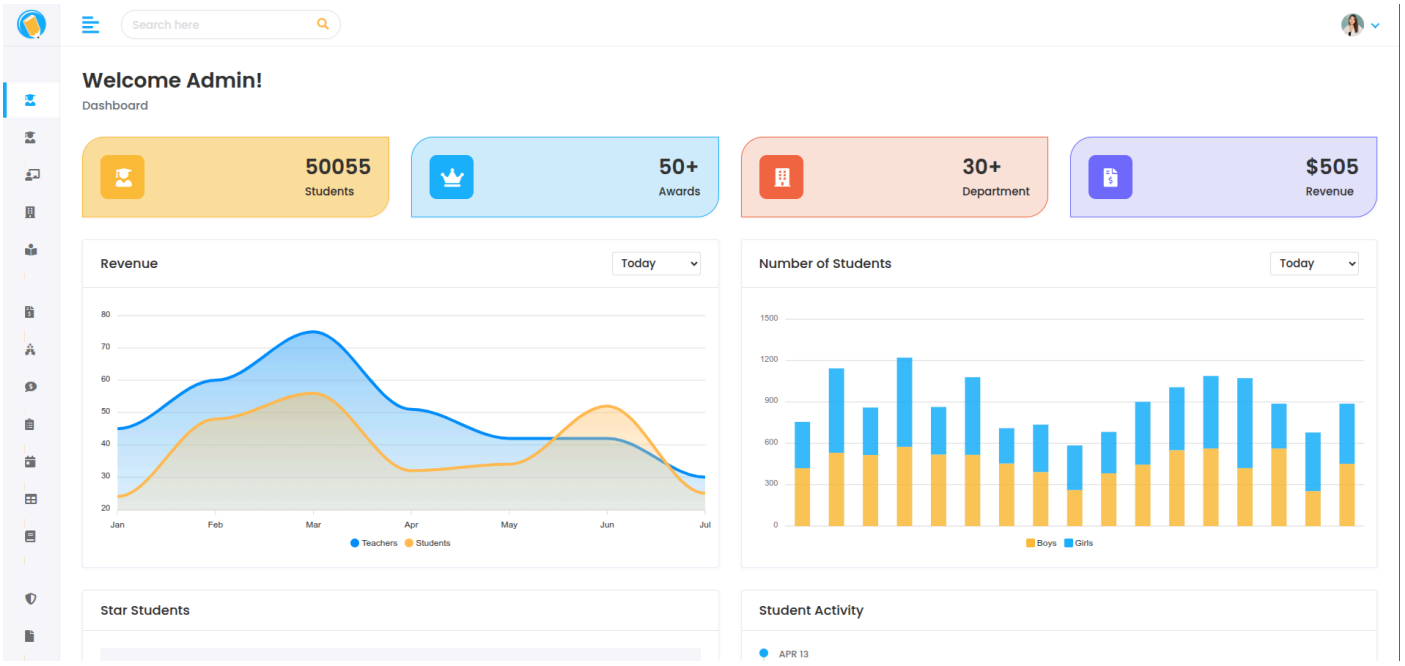
4.1.Authentification et Tableaux de Bord (Dashboards)

La sécurité et la personnalisation de l'espace de travail sont assurées par un système d'authentification robuste. Chaque type d'utilisateur est redirigé vers une interface qui lui est propre après la connexion.

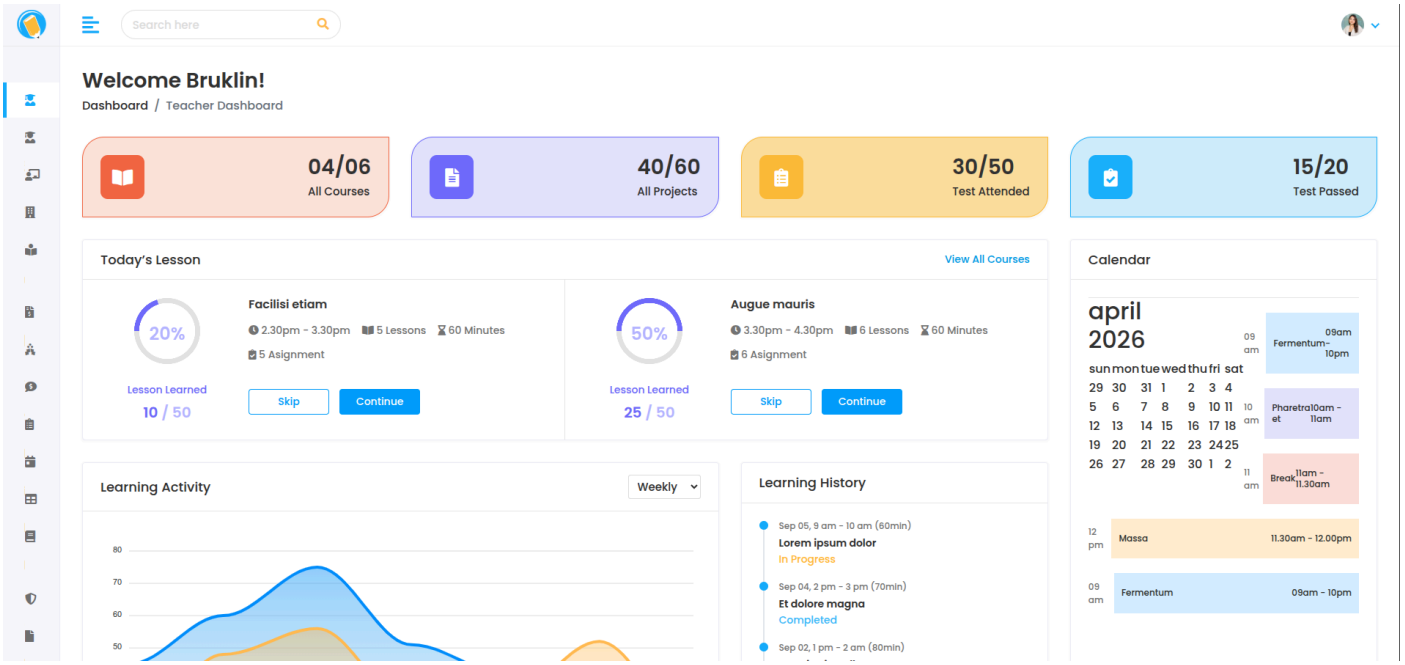
- **Page de connexion :** L'interface de connexion exige un identifiant et un mot de passe valide.



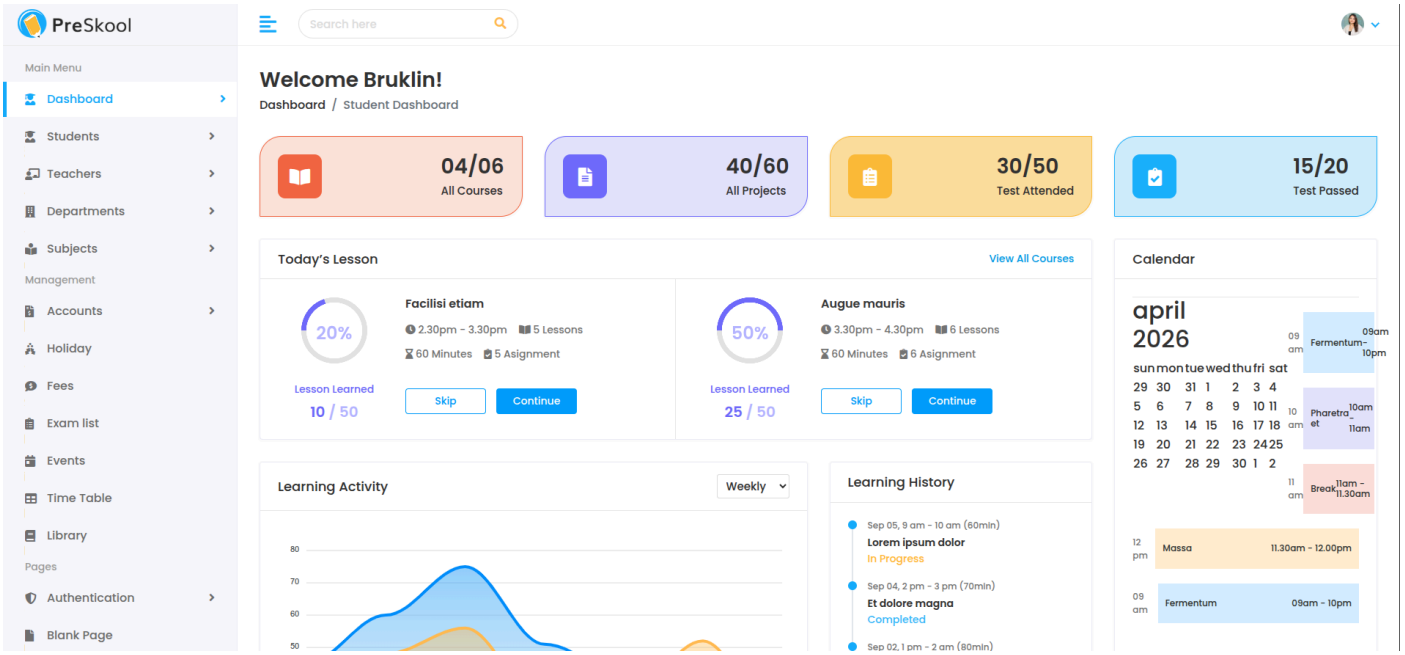
- **Tableau de Bord - Administrateur :** L'administrateur dispose d'une vue d'ensemble sur le système (statistiques globales, raccourcis vers la gestion des utilisateurs et des départements).



- **Tableau de Bord - Professeur :** L'espace enseignant met en avant les classes assignées, l'emploi du temps spécifique au professeur, et la gestion des notes/absences.



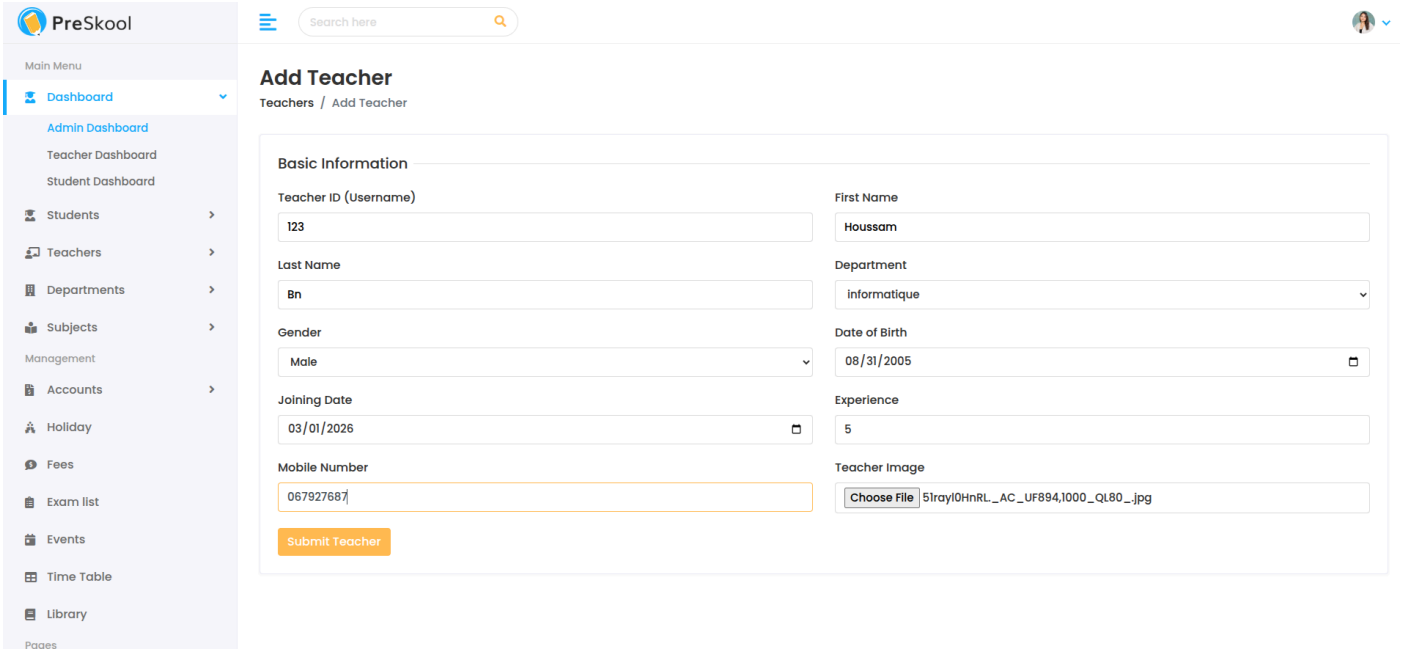
- **Tableau de Bord - Étudiant :** L'étudiant a accès à un espace en lecture seule concernant son propre cursus : ses notes, son emploi du temps et les annonces de ses départements.



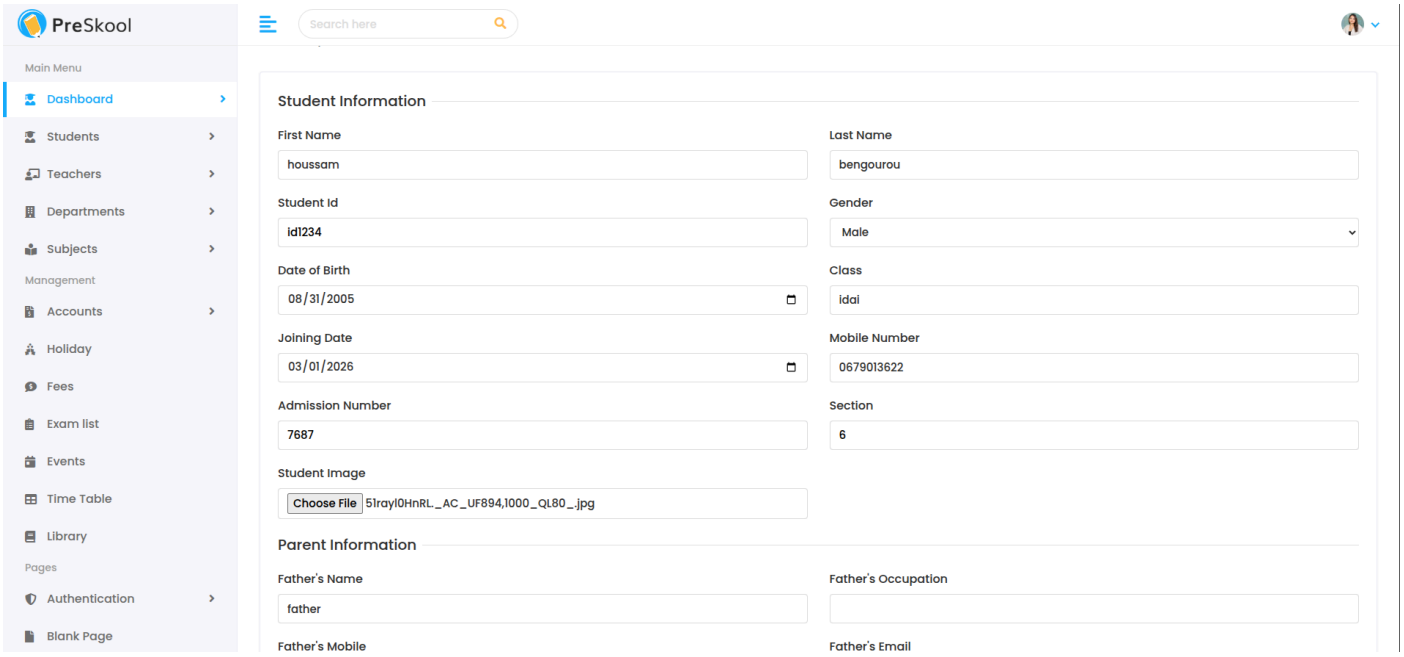
4.2.Gestion des Utilisateurs

Ce module, principalement accessible à l'administration, permet de gérer les acteurs de l'établissement scolaire. L'utilisation des formulaires permet une saisie rapide et sécurisée des données en base.

- **Création d'un Profil Professeur :** Le formulaire de création permet de renseigner les informations personnelles, les spécialités et d'assigner les rôles appropriés.



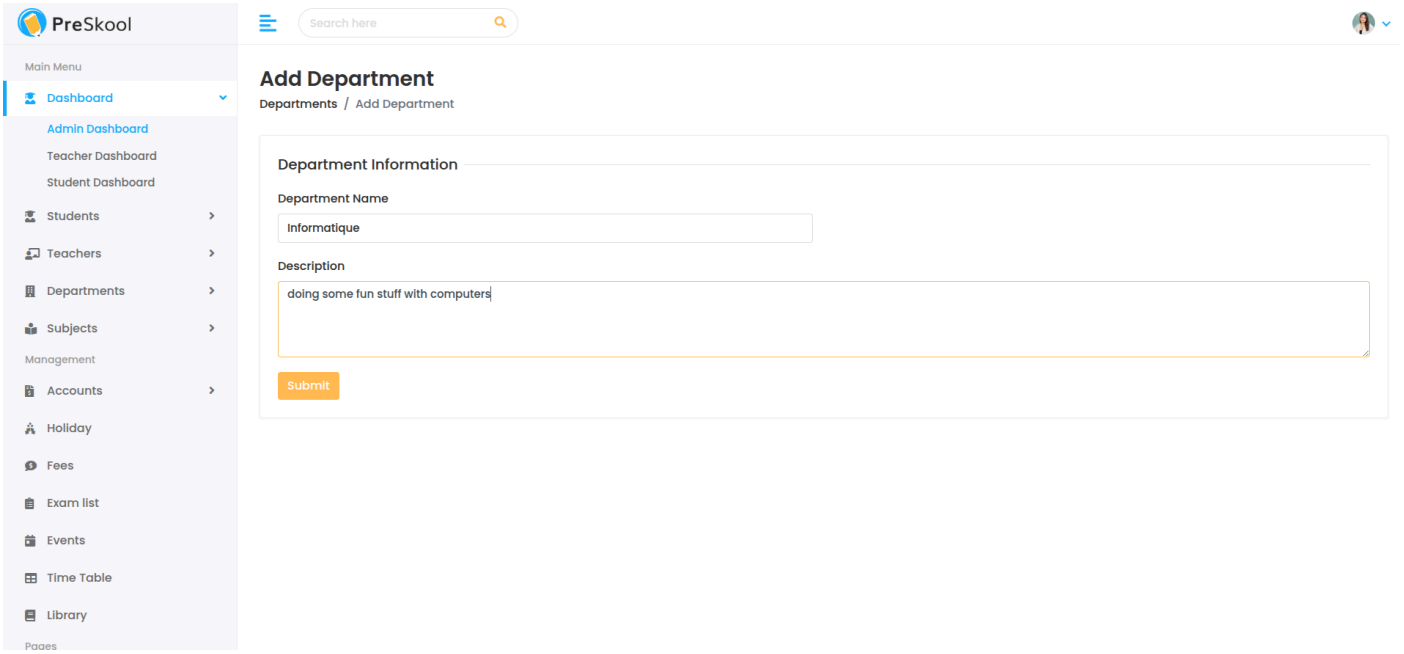
- **Création d'un Profil Étudiant :** L'interface d'inscription d'un étudiant inclut ses données d'état civil, son matricule et son affectation à une classe ou un niveau spécifique.



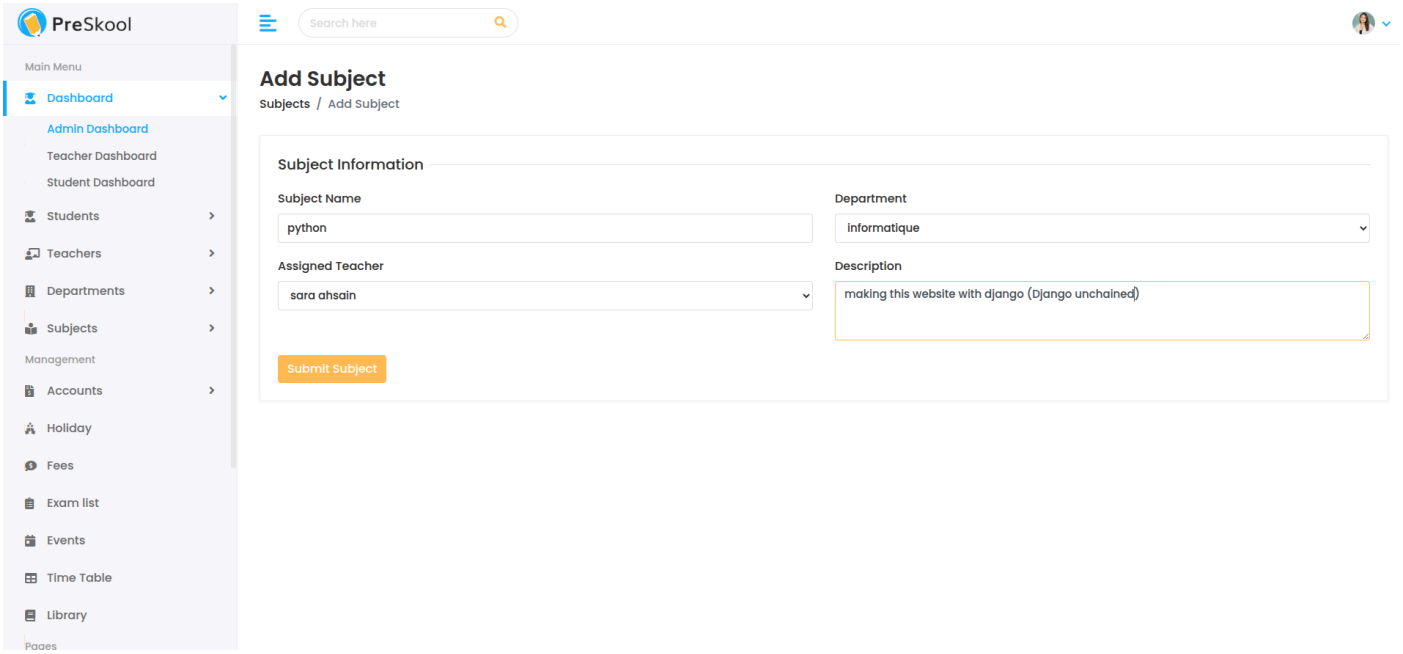
4.3.Scolarité (Départements et Matières)

La structuration académique de l'établissement est gérée dans cette section. Elle permet de définir la hiérarchie de l'école.

- **Gestion des Départements :** Interface permettant de créer et de lister les différents départements de l'établissement (ex: Informatique, Mathématiques, Langues).



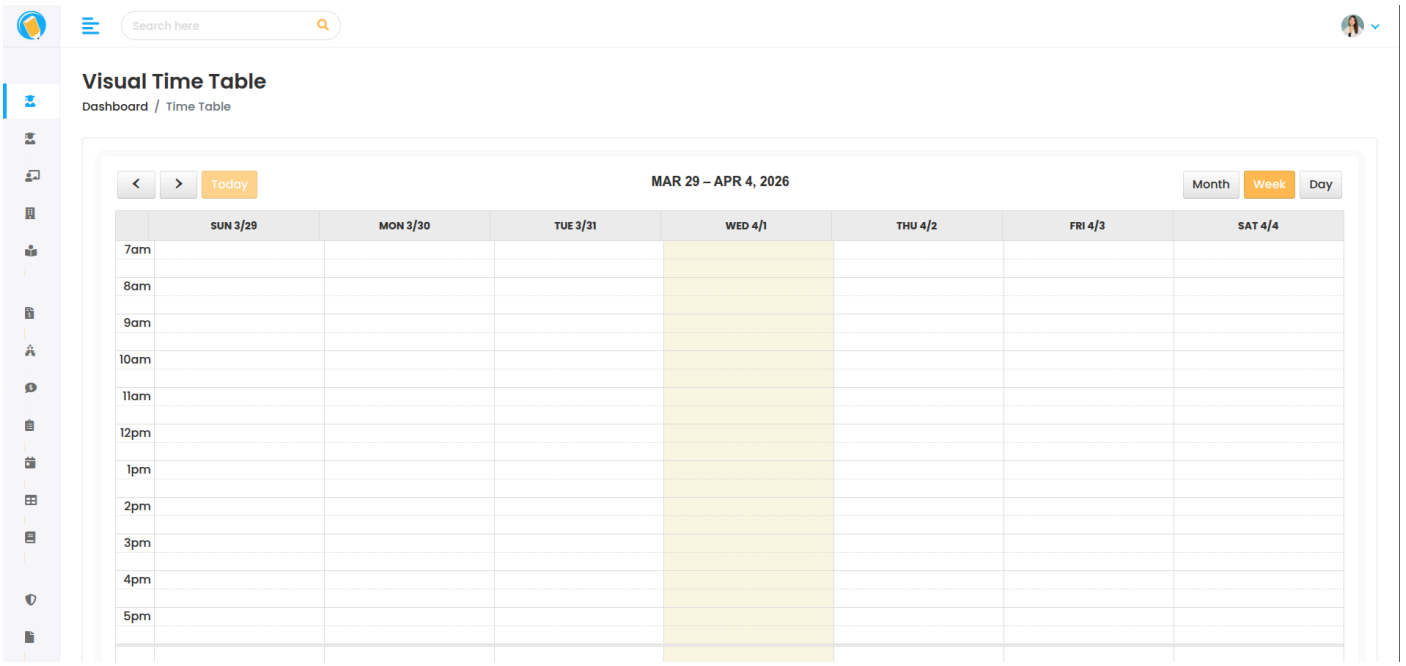
- **Gestion des Matières (Subjects) :** Chaque matière est créée et rattachée à un département, avec la possibilité de définir des coefficients ou des volumes horaires.



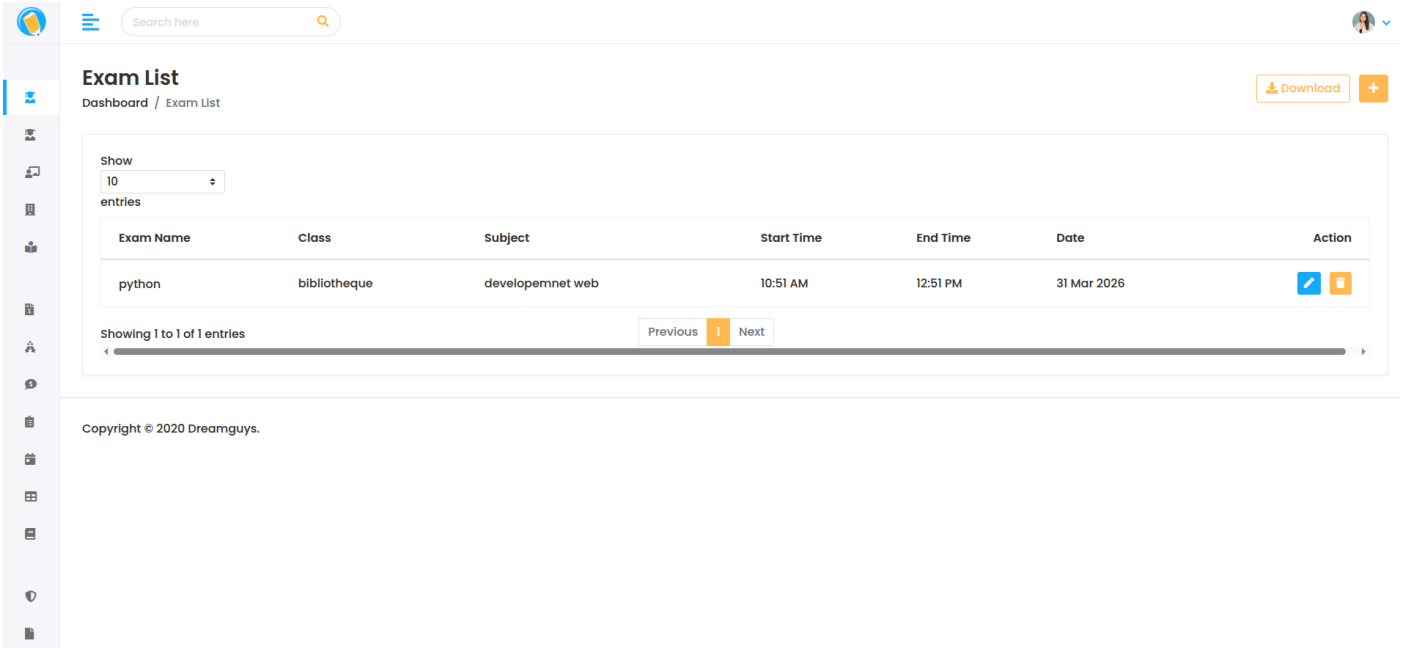
4.4.Planification et Événements

La gestion du temps est cruciale pour le bon fonctionnement de l'école. Ce module centralise les emplois du temps et le calendrier académique.

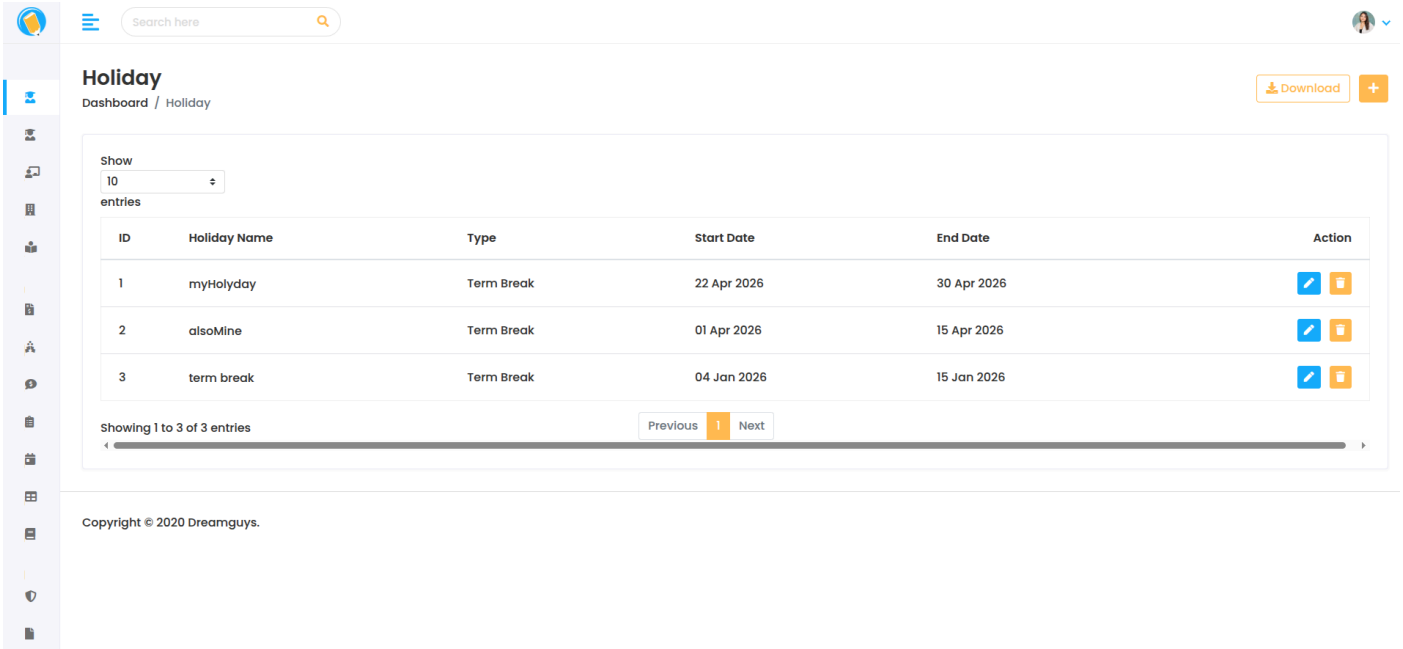
- **Emploi du Temps :** L'interface présente le planning hebdomadaire, répartissant les matières, les salles et les professeurs pour une classe donnée.



- **Planification des Examens :** Vue permettant de programmer les sessions d'examens, d'assigner des surveillants et d'informer les étudiants concernés.



- **Jours Fériés et Vacances :** Gestion du calendrier des jours chômés pour ajuster automatiquement les emplois du temps et les présences.



5.Améliorations Possibles

Bien que l'application réponde aux exigences fonctionnelles principales de gestion scolaire, plusieurs optimisations peuvent être envisagées pour enrichir l'expérience utilisateur et moderniser l'interface. Voici les axes d'amélioration prioritaires pour les futures versions du projet :

- **Chargement Asynchrone des Données (AJAX / Fetch API) :** Actuellement, la navigation et la soumission de certains formulaires nécessitent un rechargement complet de la page. L'intégration de requêtes asynchrones (via AJAX ou l'API Fetch JavaScript) permettrait de charger et de mettre à jour les tableaux de données (liste des étudiants, professeurs, départements) de manière dynamique. Cela fluidifierait considérablement l'expérience utilisateur, en particulier pour

  les administrateurs qui manipulent de grands volumes de données au quotidien.

- **Intégration d'un Emploi du Temps Visuel et Interactif :** Afin d'aller au-delà de la simple liste tabulaire pour la planification, une interface d'emploi du temps visuelle et interactive constituerait un ajout majeur (répondant ainsi aux objectifs bonus du cahier des charges). L'utilisation de bibliothèques front-end comme *FullCalendar* permettrait une gestion par "glisser-déposer" (drag-and-drop) des cours, facilitant la détection visuelle des conflits de salles ou d'horaires pour l'administration.

- **Enrichissement des Tableaux de Bord (Analytics et Graphiques) :** Les tableaux de bord actuels peuvent être poussés plus loin en intégrant de la data visualisation (à l'aide d'outils comme *Chart.js*). L'administration et les professeurs bénéficieraient grandement de graphiques dynamiques illustrant, par exemple :
  - Le taux de présence et d'absentéisme des étudiants par classe.

- La répartition des effectifs par département.
- L'évolution des moyennes générales au fil des semestres. Ces indicateurs visuels transformeraient le tableau de bord en un véritable outil d'aide à la décision.

6.Réflexion et Conclusion

Ce projet de plateforme de gestion scolaire a représenté une excellente opportunité de consolider mes compétences en développement web, en concevant une application complète de bout en bout. La construction d'un système répondant à des règles métiers réelles m'a poussé à structurer mon code de manière rigoureuse et à adopter de bonnes pratiques d'ingénierie logicielle.

Choix techniques et structuration du projet Dès la conception, j'ai opté pour une architecture modulaire, divisant le projet en plusieurs sous-applications distinctes (ex: gestion des utilisateurs, scolarité, planification). Ce choix a grandement facilité la maintenance du code et permis une séparation claire des responsabilités, rendant l'ensemble de la base de code beaucoup plus lisible et évolutive.
Le rôle central de Git dans la gestion de version Gérer un projet fragmenté en plusieurs applications aurait été extrêmement complexe sans un contrôle de version rigoureux. L'utilisation de Git a été indispensable à mon flux de travail. La possibilité de créer des branches distinctes pour chaque nouvelle "app" ou fonctionnalité m'a permis d'isoler mes développements et d'expérimenter sans risquer de casser la branche principale (*main* ou *master*). En cas d'erreur ou de conflit entre les différentes parties du projet, l'historique des commits m'a offert un filet de sécurité pour revenir facilement à une version stable.
Défis techniques : Les modèles utilisateurs personnalisés Le défi technique le plus significatif du projet a été l'implémentation et la gestion

des utilisateurs. Le modèle standard d'authentification étant insuffisant pour une école, j'ai dû créer un système de modèles utilisateurs personnalisés.

La principale difficulté a résidé dans la liaison de ce compte de base avec les profils spécifiques (Étudiant et Professeur). Il a fallu mettre en place des relations (généralement de type *One-to-One*) robustes dans la base de données, tout en s'assurant que lors de la création d'un utilisateur depuis un formulaire, le profil associé (avec son matricule, sa classe ou sa spécialité) soit généré simultanément et sans erreur d'intégrité. Surmonter ce défi m'a permis de comprendre en profondeur la gestion relationnelle des bases de données et a été la clé pour implémenter un système d'autorisations solide, garantissant que chaque acteur n'accède qu'au tableau de bord qui lui est destiné.

Bilan:

En somme, au-delà de la réussite technique de la plateforme, ce projet a affiné ma méthodologie de travail. La résolution des problèmes architecturaux et la rigueur imposée par les outils de versionnage ont fait de ce développement une expérience particulièrement formatrice.

Démonstration vidéo

Voici le lien de dossier contient le vidéo démonstrative:

https://drive.google.com/drive/folders/1nFgcNJ5xhro68MbJU91F94pFjnd7YabO?usp=drive_link